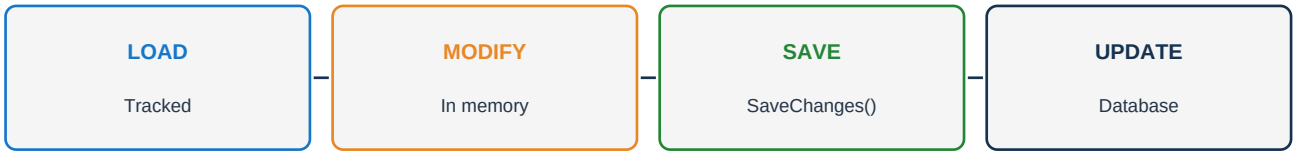
Updating Data in EF Core

Visual Lesson Summary • Corrected code formatting and line breaks

1. Core Idea

EF Core does not update the database immediately when a C# property changes. It tracks the entity, detects changes, and sends SQL when **SaveChanges()** is called.



2. Tracked vs. No-Tracking

Approach	Behavior	Result
AsNoTracking()	Entity is not tracked by this DbContext.	Editing it and calling SaveChanges() alone does not persist the edit.
Tracked query	Entity is tracked; EF can detect property changes.	SaveChanges() can generate and execute UPDATE SQL.

3. Bad Approach — AsNoTracking()

```
var student = context.Students
        .AsNoTracking()
        .FirstOrDefault(s => s.StudentId == 1);

if (student != null)
{
        student.FirstName = "UpdatedName";
        int affectedRows = context.SaveChanges();
}
```

The object changes in memory, but this DbContext is not tracking it. SaveChanges() has no tracked modification to persist. A detached entity can be attached or explicitly marked for update through other APIs, but that is not done in this example.

4. Good Approach — Tracked Entity

```
var student = context.Students
        .FirstOrDefault(s => s.StudentId == 1);

if (student != null)
{
        student.FirstName = "Ali Updated";
        int affectedRows = context.SaveChanges();
}
```

Because the entity is tracked, EF Core detects the changed property and SaveChanges() can send an UPDATE statement.

5. Example Generated SQL

```
UPDATE [Students]
SET [FirstName] = N'Ali Updated'
WHERE [StudentId] = 1;
```

EF Core often updates only properties detected as modified. Exact SQL can vary by provider, mapping, and scenario.

6. SQL Preview vs. Runtime Logging

Tool	Purpose
ToQueryString()	Shows the SQL representation of an IQueryable query (here, the SELECT query) before execution.
Runtime logging	Shows SQL commands actually executed, including the UPDATE generated by SaveChanges().

7. Important Notes

• SaveChanges() returns the number of state entries written; it does not necessarily equal the number of SQL statements in every scenario.
• EnableSensitiveDataLogging() may expose parameter values. Use carefully and avoid unsafe production logging.
• AsNoTracking() is useful for read-only queries; for this simple tracked-update pattern, do not use it.

Mental anchor: Load a tracked entity → modify it → call SaveChanges(). No tracking means no automatic update from a property edit alone.